

Rechnerstrukturen Teil II

Mehrprozessorsysteme

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12

Mehrprozessorsysteme

*“The turning away from the conventional organization came in the middle 1960s, when the **law of diminishing returns** began to take effect in the effort to increase the operational speed of a computer ... Electronic circuits are ultimately limited in their speed of operation by the speed of light ... and many ... were already operating in the nanosecond range.”*

W. Jack Bouknight et al.: The Illiac IV System (1972)

*“... sequential computers are approaching a **fundamental physical limit** on their potential power. Such a limit is the speed of light ...”*

Angel L. DeCegama: The Technology of Parallel Processing (1989)

Mehrprozessorsysteme: Einführung

Beobachtung:

Einschätzung, dass Fortschritte bei Uni-Prozessor-Architekturen sich ihrem Ende nähern

Allerdings:

1985-2000 höchste Leistungssteigerung bei Uni-Prozessoren
(seit der Einführung transistorbasierter Rechner Ende der 1950er)

- Triebkraft: Verwendung/Technologie von Mikroprozessoren

Trotzdem:

Bedeutung paralleler Prozessoren wächst und wird in der Zukunft weiter steigen

Mehrprozessorsysteme: Einführung II

- Größte Hürde für MP-Einsatz: *Software*
 - Race Conditions (Wettlaufsituationen)
 - Deadlocks
 - Synchronisationsaufwand
- Stetiger Fortschritt
 - Groß bei Server-Anwendungen: häufig “natürlicher” Parallelismus, z.B. durch unabhängige Anfragen
 - Problematischer bei Desktop-Systemen/Anwendungen: Oft interaktiv und historisch aus single-core gewachsen
 - Fortschritte bei Programmiersystemen (z.B. OpenMP)

Mehrprozessorsysteme: Einführung III

- Bis etwa 2000: Leistungssteigerung hauptsächlich durch höhere Taktfrequenzen
- Danach: Physikalische Grenzen (Wärmeentwicklung, Leckströme)
- Moor'sches Gesetz gilt weiter, aber anders:
 - Integrationsdichte (Transistoren pro Chip) wächst weiterhin an
 - Dadurch größere Caches und mehr Rechenkerne möglich
- Vorstoß von Multiprozessoren in breitere Märkte
 - Früher: Nur in Supercomputern oder Servern
 - Heute: Standard in Laptops, Smartphones, etc.
- Zentrale Eigenschaft: Parallelität
 - Schon immer wichtigste algorithmische Triebkraft bei der Leistungssteigerung (siehe Pipelining)
 - Multiprozessorsysteme setzen dieses Prinzip fort (auf anderer Granularitätsstufe)
- Mehrprozessorsysteme werden in Zukunft weiter an Attraktivität gewinnen!

Moore's Law

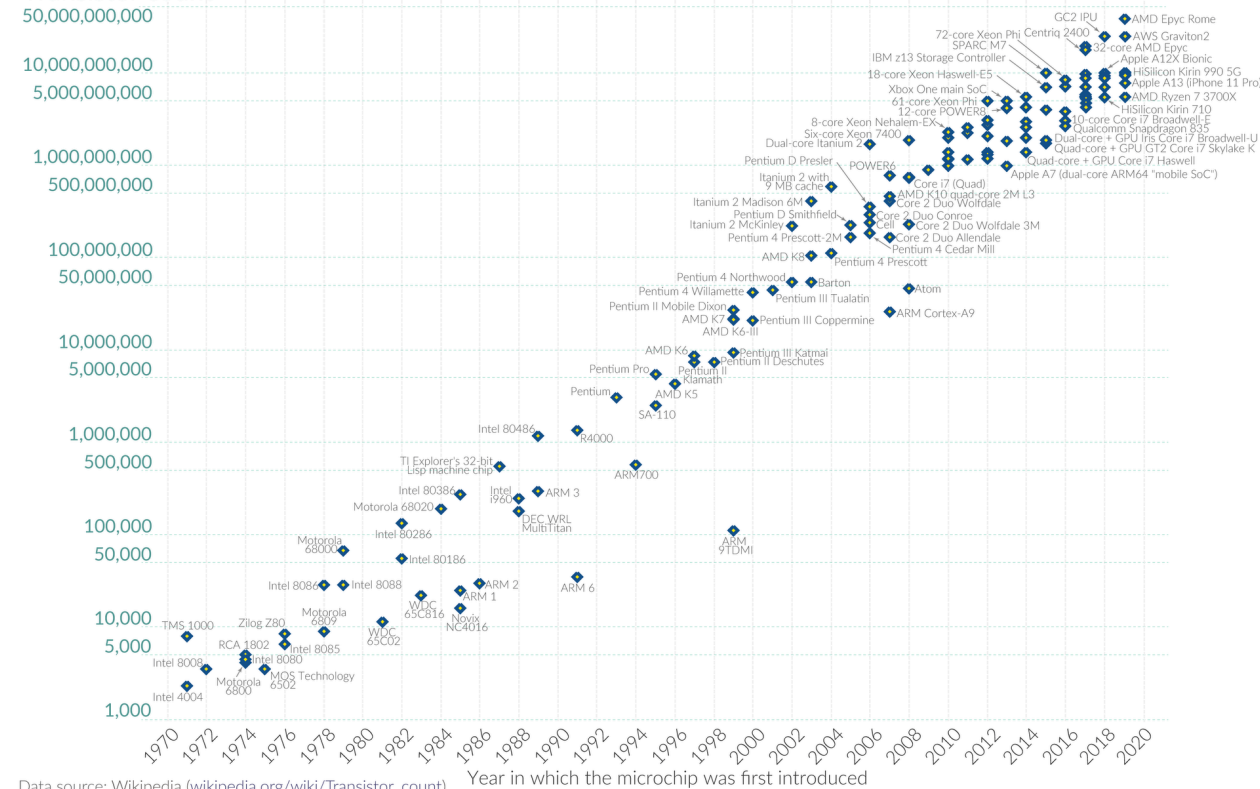
“The number of transistors in an integrated circuit (IC) **doubles** about every two years.”

Moore's Law: The number of transistors on microchips doubles every two years

Our World
in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count

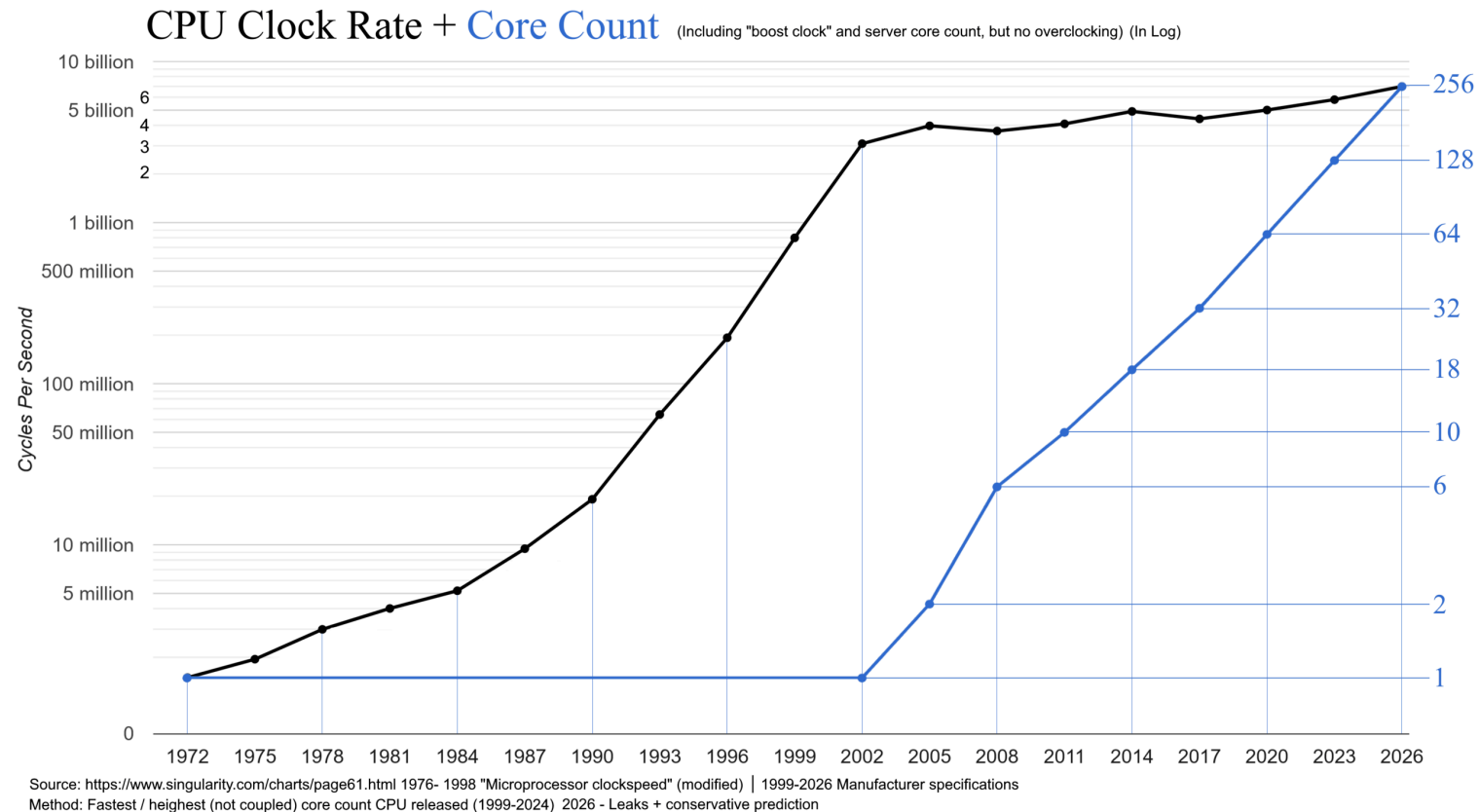


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)
OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under [CC-BY](#) by the authors Hannah Ritchie and Max Roser.

Dennard Scaling

“As transistors get smaller, their **power density stays constant**, so that the power use stays in proportion with area.”



Zusammenbruch des Gesetzes in ~2005!

Taxonomie von Mehrprozessorsystemen

- Idee der Leistungssteigerung durch Parallelität wird bereits seit den Anfängen der Rechnerentwicklung verfolgt
- Was läuft eigentlich parallel? Befehle/Daten/Beides?

Taxonomie nach Flynn (1966):

- Single instruction, single data (SISD)
- Single instruction, multiple data (SIMD)
- Multiple instruction, single data (MISD)
- Multiple instruction, multiple data (MIMD)

Taxonomie von Mehrprozessorsystemen II

Single instruction, single data (SISD):

- Entspricht “klassischem/normalem” Uni-Prozessor-System

Single instruction, multiple data (SIMD):

- Gleicher Befehl(-sstrom) wird von mehreren Prozessoren auf unterschiedlichen Daten(strömen) ausgeführt
- Eigene Datenspeicher pro Prozessor, aber gemeinsamer Befehlsspeicher und Kontrolleinheit (zum Holen/Ausgeben der Befehle)

Taxonomie von Mehrprozessorsystemen III

Multiple instruction, single data (MISD):

- Mehrere Verarbeitungsknoten, in Matrix angeordnet, bearbeiten verschiedene Befehlsströme
- Ein Datenstrom durchläuft Berechnungsmatrix
- Bezeichnet als “systolisches Array”

Multiple instruction, multiple data (MIMD):

- Jeder Prozessor bearbeitet eigenen Befehls- und eigenen Datenstrom
- Verwendung “normaler” Mikroprozessoren möglich!

- Verbreitetstes Architektur-Schema paralleler Rechner

Taxonomie von Mehrprozessorsystemen: Diskussion

- Grobe Kategorisierung $\Rightarrow$ Oft hybride Formen
- SIMD und MISD nur für spezielle Berechnungen geeignet
- Viele frühe MPs nach SIMD-Prinzip, seit 1990er praktisch verschwunden (Ausnahme: Vektorrechner)
- Systolische Arrays (MISD) erhalten mehr Aufmerksamkeit für KI-Beschleunigung
- MIMD ist vorherrschende Architektur für “*general purpose*” Mehrprozessorsysteme
 - Große Flexibilität
 - sowohl ganzes Mehrprozessorsystem für eine Anwendung, als auch viele unterschiedliche Prozesse möglich
 - Können von Kosten-Leistungs-Vorteilen gängiger Mikroprozessoren profitieren
 - praktisch alle heutigen MIMD-Systeme mit gleichen Mikroprozessoren wie Server/Desktops realisiert!

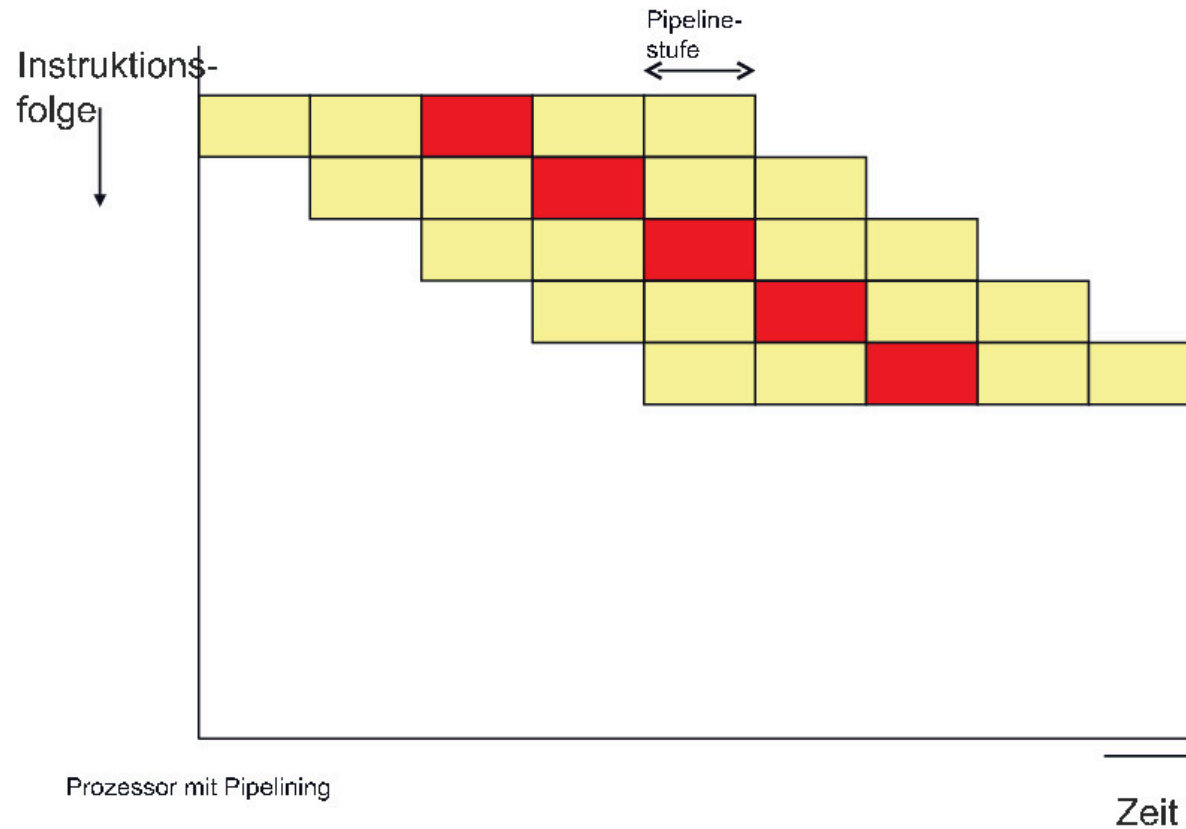
Parallelität

Welche Arten der Parallelität kennen wir?

- Pipelining
- Superskalarität
- Vektorprozessoren
- Very Long Instruction Word (VLIW)

Parallelität: Pipelining

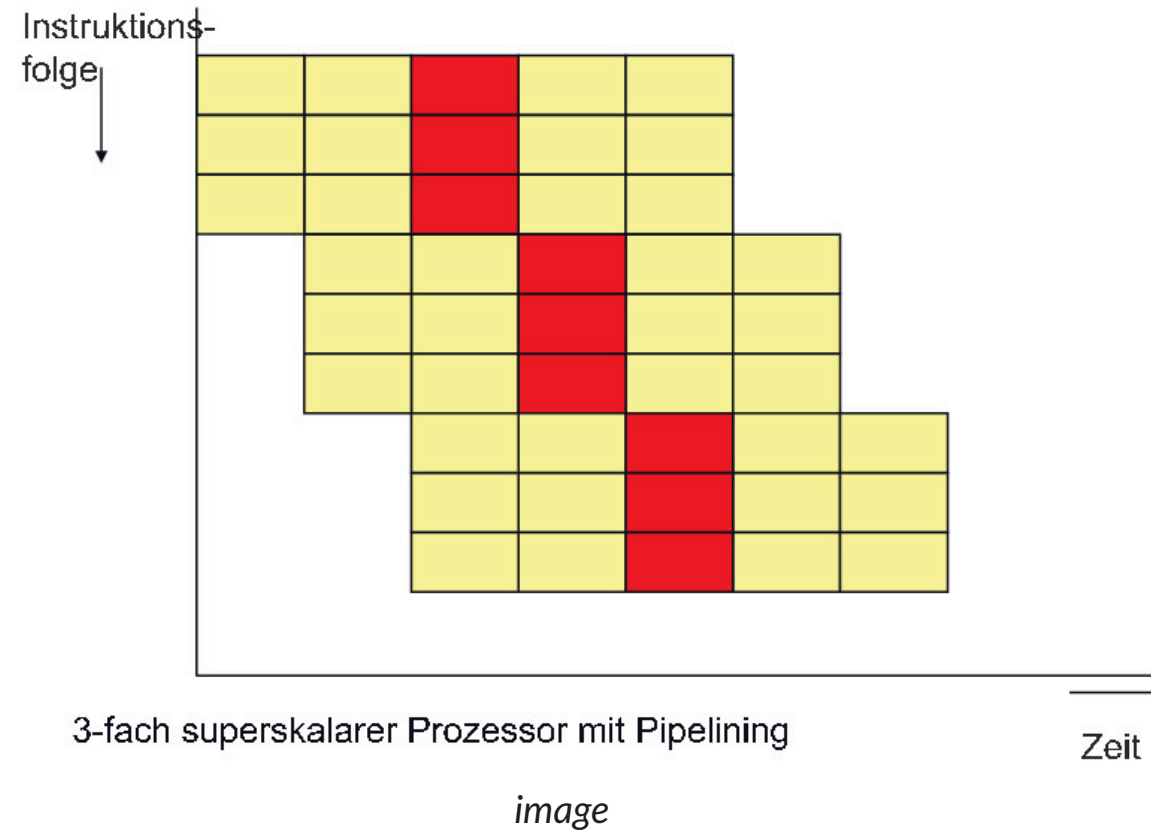
Grundidee: Aufteilung der Instruktionsausführung in mehrere Stufen



image

Parallelität: Superskalarität

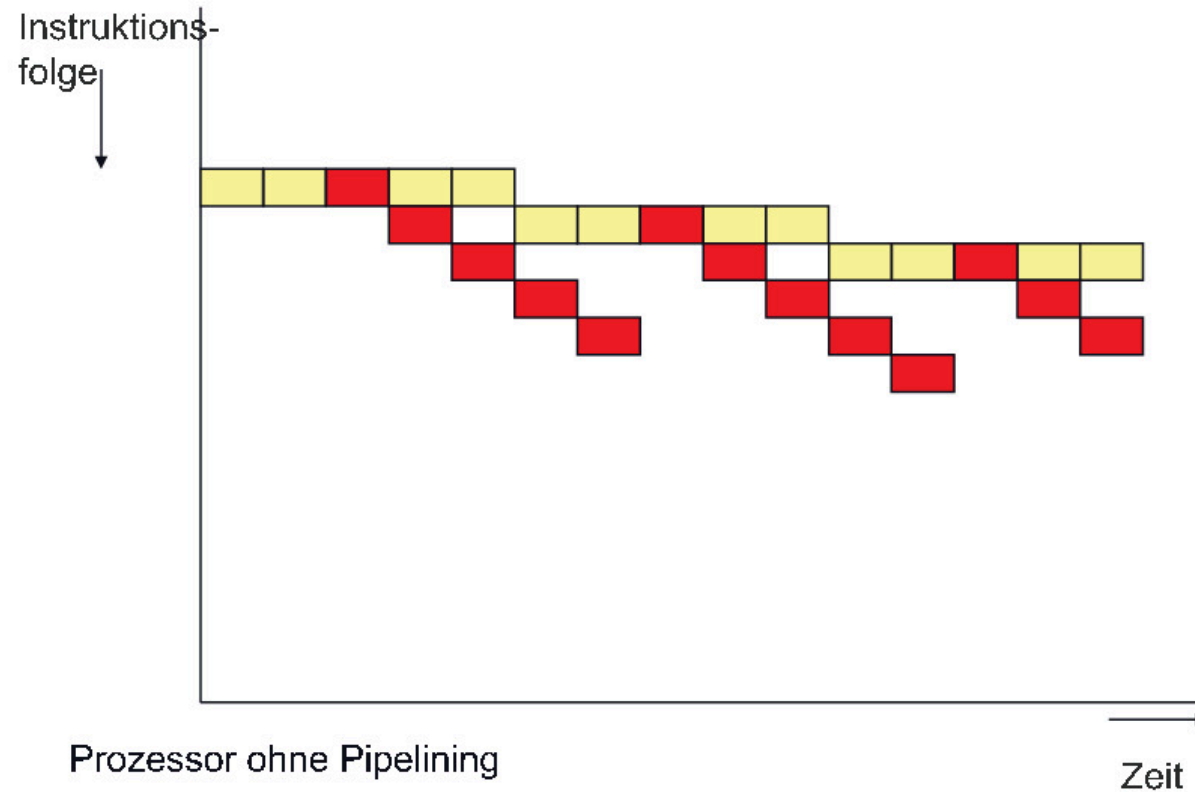
Grundidee: Mehrere Instruktionen pro Takt ausführen



Quelle: R. Yahyapour, ehem. TU Dortmund

Parallelität: Vektorprozessoren

Grundidee: Ein Befehl verarbeitet viele Daten gleichzeitig

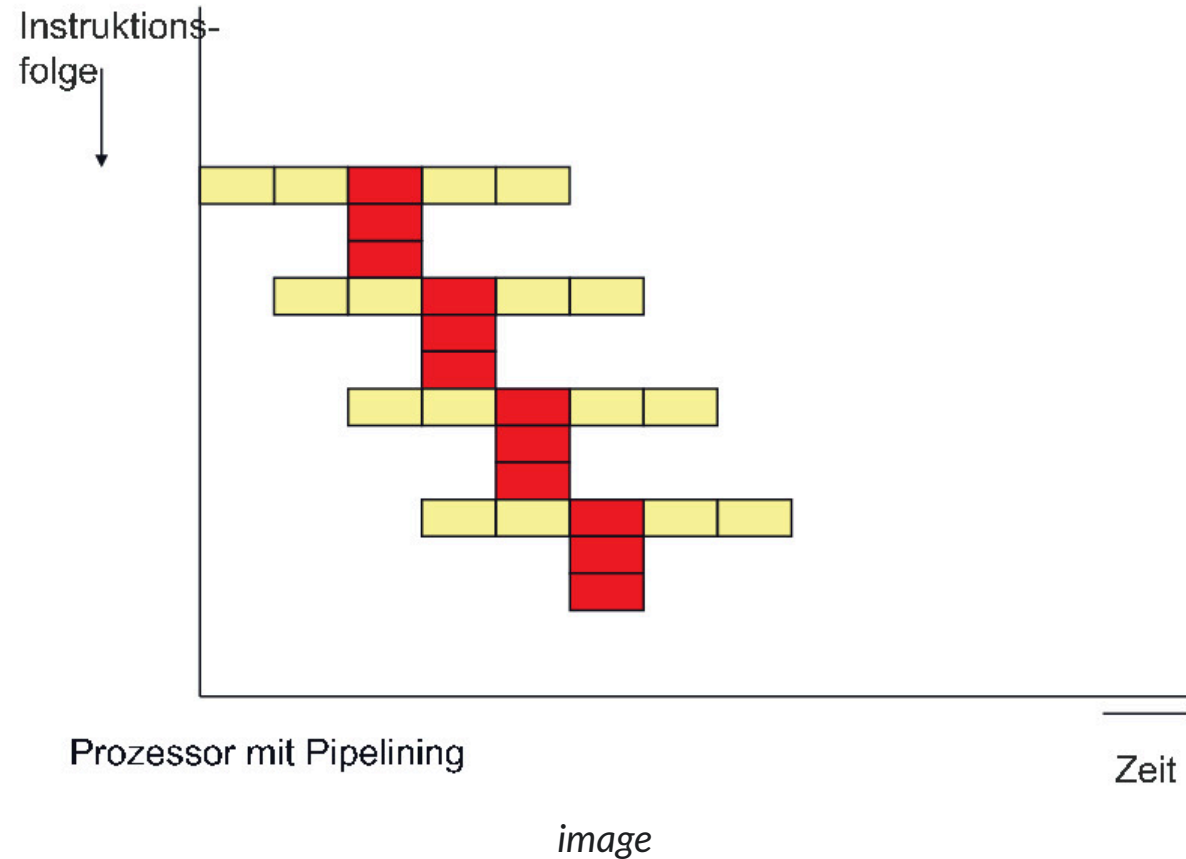


image

Quelle: R. Yahyapour, ehem. TU Dortmund

Parallelität: Very Long Instruction Word (VLIW)

Grundidee: Parallelität wird vom Compiler festgelegt



Quelle: R. Yahyapour, ehem. TU Dortmund

Amdahl'sches Gesetz

- Intuitive Erwartung: Doppelt so viele Prozessoren → Doppelte Leistung
- Realität:
 - Nur der parallelisierbare Anteil profitiert
 - Kosten der notwendigen Kommunikation (gemeinsame Daten, Synchronisation, etc.)

Welchen Leistungsgewinn erzielt ein N-Prozessor-System gegenüber einem Uni-Prozessor?

$$S_{\text{tot}} = \frac{\text{Laufzeit ohne Parallelisierung}}{\text{Laufzeit mit Parallelisierung}}$$

⇒ Gesetze von Amdahl und Gustafson machen darüber Aussagen!

Amdahl'sches Gesetz II

Zentrale Frage: Wieviel schneller ist Programm mit Parallelisierung?

$$S_{\text{tot}} = \frac{1}{(1-P) + \frac{P}{S}} = \frac{1}{(1-P) + \frac{P}{N}}$$

- P : Anteil des parallelisierbaren Codes der Anwendung
- Speedup S , der durch paralleles Rechnen erreicht wird
 - vereinfachende Annahme: Parallelisierung auf parallelisierbarem Code [Anteil P] perfekt
 - $\Rightarrow$ Speedup $S = N = \text{Anzahl der Prozessoren}$

Amdahl'sches Gesetz III

$$S_{\text{tot}} = \frac{1}{(1-P) + \frac{P}{N}}$$

- Betrachten Beispiel (jeweils $N = 64$ Prozessoren):
 - 80% parallelisierbarer Code $\Rightarrow$ Speedup ca. 4,7
 - 50% parallelisierbarer Code $\Rightarrow$ Speedup ca. 2
 - 20% parallelisierbarer Code $\Rightarrow$ Speedup ca. 1,25
- Maximaler Speedup bei beliebig vielen Prozessoren:

$$\lim_{N \rightarrow \infty} S_{\text{tot}} = \frac{1}{(1-P)}$$

Das Gesetz von Gustafson (& Barsis)

Zentrale Frage: Wieviel langsamer ist Programm ohne Parallelisierung?

$$S'_{\text{tot}} = (1 - P) + P \cdot N$$

- Betrachten Beispiel (wieder jeweils $N = 64$ Prozessoren):
 - 80% parallelisierbarer Code $\Rightarrow$ Speedup ca. 51,4
 - 50% parallelisierbarer Code $\Rightarrow$ Speedup ca. 32,5
 - 20% parallelisierbarer Code $\Rightarrow$ Speedup ca. 13,6
- Wieso dieser (scheinbare) Widerspruch zum Amdahl'schen Gesetz?
 - Amdahl: Problemgröße ist fest und wir beschleunigen ein konkretes Programm

- Gustafson: Laufzeit bleibt konstant und die Problemgröße wächst mit der Prozessoranzahl

Amdahl vs. Gustafson

Amdahl's Gesetz (feste Problemgröße) sagt etwa:

*“Suppose a car is traveling between two cities 60 miles apart, and has already spent one hour traveling half the distance at 30 mph. No matter how fast you drive the last half, it is **impossible** to achieve 90 mph average **before reaching the second city**. Since it has already taken you 1 hour and you only have a distance of 60 miles total; going infinitely fast you would only achieve 60 mph.”*

Gustafson's Gesetz (konstante Laufzeit) sagt etwa:

*“Suppose a car has already been traveling for some time at less than 90mph. Given enough time and distance to travel, the car's average speed **can always eventually reach 90mph**, no matter how long or how slowly it has already traveled. For example, if the car spent one hour at 30 mph, it could achieve this by driving at 120 mph for two additional hours, or at 150 mph for an hour, and so on.”*

(Quelle: Wikipedia)

Zusammenfassung

- Gesetz von Moore: Verdopplung Transistorenanzahl alle 2 Jahre
- Dennard Scaling: Konstante Leistungsdichte
- Taxonomie nach Flynn: SISD, SIMD, MISD, MIMD
- Parallelität: Pipelining, Superskalarität, Vektorprozessoren, VLIW
- Ahmdahl'sches Gesetz vs Gesetz von Gustafson: Leistungsgewinn von Multiprozessoren

Viel Erfolg beim Lernen und bei der Klausur! :)

